

UNIVERSITÀ POLITECNICA DELLE MARCHE
INGEGNERIA INFORMATICA E DELL'AUTOMAZIONE

MOSHPIT

Progetto di gestione eventi con Laravel 12



Corso di
TECNOLOGIE WEB

Anno accademico 2025-2026

Studenti:

Lorenzo Califano

Andrea Carnevale

Chiara Carlomagno

Francesca Grottola

Professore:

Alessandro Cucchiarelli



Dipartimento di Ingegneria dell'Informazione

Indice

1	Introduzione	2
2	Riferimenti	3
3	Contributo	3
4	Descrizione del sito	4
4.1	Area pubblica e catalogo eventi	4
4.2	Ricerca e filtro degli eventi	4
4.3	Area cliente	5
4.4	Acquisto biglietti	5
4.5	Partecipazioni e AJAX	5
4.6	Area organizzatore	6
4.7	Sconto last-minute	6
4.8	Area amministratore	6
4.9	Database, migration e seeder	7
4.10	View Blade, CSS e JavaScript	7
5	Organization chart	7
6	Soluzioni adottate	12
6.1	Gestione degli eventi futuri e mantenimento dello storico	12
6.2	Ricerca testuale con wildcard	13
6.3	Funzionalità “Parteciperò” con jQuery e AJAX	14
6.4	Relazione tra eventi e partecipazioni	16
6.5	Sconto last-minute centralizzato nel model Evento	17
6.6	Controllo JavaScript dello sconto last-minute	18
6.7	Gestione dello sconto in modifica evento	19
6.8	Gestione immagini eventi	20
6.9	Conferma acquisto	21
6.10	Metodi di pagamento comuni	22
6.11	Protezione CSRF e metodo DELETE	23
6.12	Configurazione database	24
6.13	Organizzazione CSS senza framework esterni	24
6.14	Home pubblica con iframe Google Maps	25
7	Conclusione	27

1 Introduzione

Il presente documento descrive l'analisi, la progettazione e l'implementazione di **MOSH PIT**, una piattaforma web integrata dedicata alla promozione e alla commercializzazione di eventi sul territorio nazionale. Lo sviluppo dell'applicazione si colloca nell'ambito del progetto didattico intracorso per l'insegnamento di *Tecnologie Web* (Anno Accademico 2025-2026) presso l'Università Politecnica delle Marche.

Obiettivi Didattici

L'attività progettuale è finalizzata all'acquisizione di competenze pratiche e metodologiche necessarie per la realizzazione di moderne architetture web. In particolare, il percorso di sviluppo ha permesso di approfondire e integrare le seguenti tecnologie:

- **Sviluppo Front-end (Lato Client):** Uso del linguaggio HTML per la strutturazione semantica dei contenuti, dei fogli di stile CSS per la definizione di layout responsive e coerenti, e di JavaScript e jQuery per la gestione asincrona degli eventi (AJAX) e la manipolazione dinamica del Document Object Model (DOM).
- **Sviluppo Back-end (Lato Server):** Studio, configurazione e deployment del framework **Laravel 12**. L'utilizzo di Laravel ha consentito di applicare l'architettura MVC (Model-View-Controller), gestire in sicurezza la persistenza dei dati tramite l'ORM Eloquent, implementare i sistemi di routing e strutturare i meccanismi di autenticazione e autorizzazione per i diversi livelli di utenza.

Il Contesto del Progetto

Il sistema **MOSH PIT** è configurato come un portale gestito da un'azienda di servizi che funge da intermediario tra gli organizzatori di eventi e il pubblico finale. Dal punto di vista funzionale, l'applicazione divide l'esperienza d'uso in base a classi di utenza ben definite:

1. **Amministratore:** Ha il controllo totale della piattaforma, si occupa della presentazione dei dati aziendali (loghi, recapiti, contratti di fornitura dei servizi) e supervisiona le adesioni degli organizzatori.
2. **Organizzatori:** Previa registrazione e autorizzazione, dispongono di una dashboard per inserire, modificare e monitorare il catalogo dei loro eventi.
3. **Utenti/Visitatori:** Possono consultare liberamente il catalogo, visualizzare i dettagli logistici (date, orari, location, programmi) ed usufruire dei servizi di acquisto previsti dal sistema.

2 Riferimenti

All'interno di questa sezione vengono riportati i dati identificativi del gruppo di lavoro e dei rispettivi componenti che hanno preso parte alla progettazione e allo sviluppo della piattaforma.

Cognome	Nome	Matricola
Califano	Lorenzo	S1114896
Carlomagno	Chiara	S1115047
Carnevale	Andrea	S1115004
Grottola	Francesca	S1115338

Componenti del gruppo di lavoro

- **Identificativo Gruppo:** Gruppo n. 3
- **Insegnamento:** Tecnologie Web
- **Corso di Laurea:** Ingegneria Informatica e dell'Automazione
- **Ateneo:** Università Politecnica delle Marche

3 Contributo

Lo sviluppo della piattaforma MOSHPIT è stato affrontato adottando una metodologia di lavoro strettamente cooperativa. Fin dalle prime fasi di analisi dei requisiti e progettazione dell'architettura, il gruppo ha prediletto un confronto continuo, finalizzato a una condivisione costante delle scelte tecnologiche e implementative.

Questo approccio ha permesso di mantenere i carichi di lavoro e le responsabilità decisionali pressoché identici tra tutti i membri del team per l'intera durata del progetto. Avanzando nello sviluppo di pari passo e in modo bilanciato, non si sono registrati squilibri o disallineamenti nelle attività individuali.

Di conseguenza, l'apporto complessivo alla realizzazione della documentazione e dell'applicazione web in Laravel 12 è da considerarsi equamente suddiviso come segue:

Studente	Contributo Percentuale
Lorenzo Califano	25%
Chiara Carlomagno	25%
Andrea Carnevale	25%
Francesca Grottola	25%
Totale	100%

Suddivisione del carico di lavoro

4 Descrizione del sito

Il progetto Moshpit è un'applicazione web sviluppata in Laravel per la gestione e la consultazione di eventi live. L'applicazione segue il pattern MVC: le route definiscono gli URL disponibili, i controller gestiscono la logica applicativa, i model rappresentano le entità del database tramite Eloquent e le view Blade si occupano della presentazione dei dati. Questa separazione permette di mantenere distinto il codice che recupera e modifica i dati dal codice che li visualizza nelle pagine. Le route principali sono definite in [routes/web.php](#), mentre le route relative all'autenticazione sono raccolte in [routes/auth.php](#). L'accesso alle aree riservate è protetto tramite middleware auth e controlli sui ruoli. In questo modo un utente non autenticato non può accedervi. Nelle view vengono inoltre usate direttive Blade come @auth, @guest e @can, che permettono di mostrare o nascondere parti della pagina in base allo stato di autenticazione e al ruolo dell'utente.

4.1 Area pubblica e catalogo eventi

La parte pubblica del sito è gestita principalmente dal PubblicoController. Questo controller espone la home, il catalogo eventi, il dettaglio evento, i contatti e la documentazione. La home pubblica, definita in [resources/views/pubblico/home.blade.php](#), presenta il servizio, i recapiti, la localizzazione dell'azienda e un iframe Google Maps. Il catalogo eventi è visualizzato tramite la view [resources/views/pubblico/eventi.blade.php](#). Gli eventi mostrati nella parte pubblica sono filtrati in base alla data, in modo da visualizzare solo eventi non ancora passati. La scelta di non eliminare automaticamente gli eventi passati dal database permette di mantenere uno storico utile per organizzatori e amministratori, evitando però che eventi conclusi rimangano visibili nel catalogo pubblico. Gli eventi sono rappresentati dal model Evento, definito in [app/Models/Evento.php](#). Nel model sono presenti le relazioni con acquisti e partecipazioni e i metodi relativi allo sconto last-minute. Questa scelta evita di duplicare la logica del prezzo scontato in più controller o view: il calcolo rimane centralizzato nel model e può essere riutilizzato sia durante la visualizzazione dell'evento sia durante l'acquisto. Le immagini degli eventi sono gestite tramite il campo immagine, che contiene solo il nome del file. I file immagine sono collocati nella cartella [public/images/eventi](#). In questo modo il database non contiene il file vero e proprio, ma solo il riferimento testuale all'immagine. Nel seeder viene assegnata un'immagine di default, placeholder.png, così ogni evento ha sempre un'immagine visualizzabile anche se non è stata caricata un'immagine specifica.

4.2 Ricerca e filtro degli eventi

La ricerca degli eventi è implementata lato server nel PubblicoController. La distinzione tra le funzionalità dei due livelli di utenti viene gestita controllando il ruolo. La ricerca testuale per il livello due supporta il carattere wildcard *, ammesso esclusivamente come ultimo carattere. Questo controllo viene effettuato lato server prima di costruire la query, così da evitare ricerche non conformi alle specifiche. Se il carattere * viene usato in una

posizione non valida, il sistema restituisce un errore e riporta l'utente alla form con i valori già inseriti. Nelle form viene utilizzata la funzione `old()` di Laravel, per conservare i dati inseriti dall'utente in caso di errore di validazione. Questo migliora l'usabilità perché l'utente non deve ricompilare tutti i campi dopo un errore.

4.3 Area cliente

L'area cliente è gestita dal `ClienteController`, in [app/Http/Controllers/-ClienteController.php](#). La modifica del profilo viene gestita dal metodo `aggiornaProfilo()`, che valida i dati ricevuti dalla form prima del salvataggio. I campi, una volta ricompilati vengono controllati da Laravel: nome, cognome, email, username, password e data di nascita devono rispettare regole precise prima di essere salvati nel database. La data di nascita viene gestita con un campo HTML `type="date"`. Anche se il browser può visualizzarla nel formato locale, il valore inviato al backend è nel formato `Y-m-d`, che viene poi validato da Laravel. Il sistema controlla che la data non sia futura e che non indichi un'età superiore a 120 anni. Email e username vengono verificati come univoci così da non generare mai un falso errore di duplicazione. La password viene aggiornata solo se il relativo campo viene compilato. Prima del salvataggio, la nuova password viene cifrata tramite hashing, evitando di memorizzare password in chiaro nel database.

4.4 Acquisto biglietti

L'acquisto è gestito dal metodo `acquista()` del `ClienteController`. Il metodo valida l'evento selezionato, il metodo di pagamento e la quantità richiesta. L'evento viene controllato prima dell'acquisto per verificare che esista e che non si sia già svolto. I metodi di pagamento sono comuni a tutti i clienti e sono rappresentati dal model `MetodoPagamento`, collegato alla tabella `metodi_pagamento`. Durante l'acquisto viene calcolato il prezzo effettivo del biglietto tramite il model `Evento`, così da applicare automaticamente eventuali sconti last-minute. Poi viene creato un record nella tabella degli acquisti, viene aggiornata la disponibilità residua dei biglietti e l'utente viene reindirizzato alla pagina [resources/views/cliente/-conferma-acquisto.blade.php](#). Questa pagina mostra "Grazie per il tuo acquisto" e il riepilogo dell'operazione. Tutte le form di inserimento, modifica, acquisto, eliminazione e logout utilizzano il token CSRF di Laravel tramite `@csrf`. Questo token protegge le richieste POST da invii non autorizzati provenienti da pagine esterne. Per le operazioni di cancellazione viene inoltre usato il metodo HTTP simulato tramite `@method('DELETE')`, seguendo la convenzione Laravel per distinguere le azioni distruttive dalle normali richieste POST.

4.5 Partecipazioni e AJAX

La relazione tra utenti ed eventi è gestita dalla tabella delle partecipazioni e dal model `Partecipazione`. Il model `Evento` contiene la relazione `partecipazioni()`, che permette di ottenere il numero di partecipanti tramite `withCount('partecipazioni')`. La funzione "Parteciperò" è implementata sia con fallback classico tramite form POST, sia con chiamata AJAX tramite jQuery. Il fallback è importante perché, anche se JavaScript non dovesse funzionare, la funzionalità rimane comunque utilizzabile tramite la normale richiesta POST. Quando invece

JavaScript è attivo, il file [public/js/functions.js](#) intercetta il submit della form, invia una richiesta AJAX alla route dedicata e riceve una risposta JSON dal ClienteController. La risposta JSON contiene il nuovo stato della partecipazione, il numero aggiornato di partecipanti e il testo da mostrare sul bottone. In questo modo il contatore viene aggiornato senza ricaricare l'intera pagina. Questa implementazione riprende il modello visto nel progetto laraProj6, dove jQuery e AJAX vengono usati per aggiornare dinamicamente una parte del documento HTML.

4.6 Area organizzatore

L'area organizzatore è gestita dal OrganizzatoreController, in [app/Http/Controllers/-OrganizzatoreController.php](#). L'utente di livello 3 accede a una dashboard dedicata e può gestire e modificare solo gli eventi associati al proprio account. Questo controllo viene effettuato filtrando gli eventi tramite il campo organizzatore_id,. L'organizzatore può creare, modificare ed eliminare eventi, aggiornare la quantità di biglietti disponibili e consultare le vendite. Le view dedicate si trovano nella cartella [resources/views/organizzatore](#). La creazione e la modifica degli eventi utilizzano form Blade dedicate. Anche qui la validazione principale dei dati inseriti nella form è lato server: i dati vengono controllati nel controller prima di essere salvati. Quando un organizzatore elimina un evento, l'evento viene rimosso dal database.

4.7 Sconto last-minute

Lo sconto last-minute riguarda la percentuale di sconto e il numero di giorni prima dell'evento in cui lo sconto deve attivarsi. La logica principale è contenuta nel model Evento, attraverso i metodi che verificano se lo sconto è attivo e calcolano il prezzo effettivo. Nel OrganizzatoreController viene effettuata la validazione dei dati inseriti dall'organizzatore. La percentuale deve essere compresa tra 0 e 100 e il numero di giorni di attivazione non può essere maggiore dei giorni mancanti all'evento. Lo sconto può essere impostato anche per attivarsi il giorno stesso dell'evento. È presente anche un controllo lato client in [public/js/functions.js](#), realizzato con jQuery che mostra un messaggio di errore se i valori inseriti non sono coerenti. In fase di modifica evento è stato gestito anche il caso in cui uno sconto sia stato impostato in precedenza. Se nel frattempo sono passati dei giorni, il sistema non blocca automaticamente la modifica dell'evento, a meno che l'organizzatore non stia cambiando i campi relativi allo sconto o alla data. Questo evita errori ingiustificati su valori già salvati correttamente in passato.

4.8 Area amministratore

L'area amministratore è gestita dal AmministratoreController e dalle view presenti in [resources/views/amministratore](#). Gli incassi vengono calcolati interrogando la tabella degli acquisti e sommando quantità e prezzo totale. Nella pagina di dettaglio di un'organizzazione vengono mostrati sia i dati aggregati sia il dettaglio degli incassi per singolo evento. Quando l'amministratore elimina un organizzatore, vengono eliminati anche gli eventi asso-

ciati. Le operazioni riservate all'amministratore sono protette sia dalle route sia dalle direttive Blade.

4.9 Database, migration e seeder

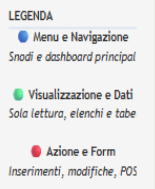
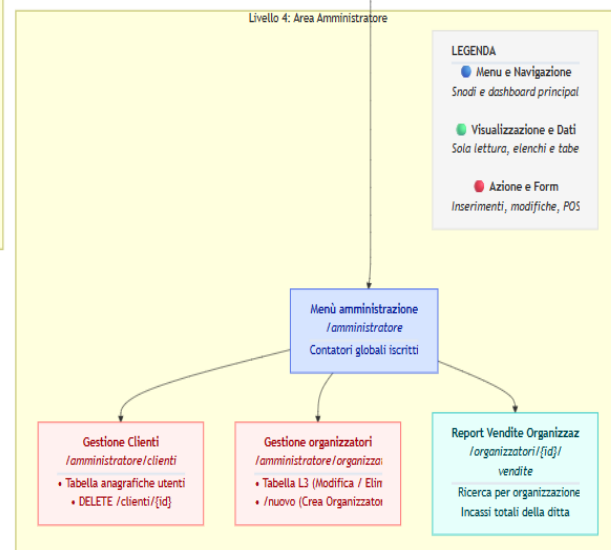
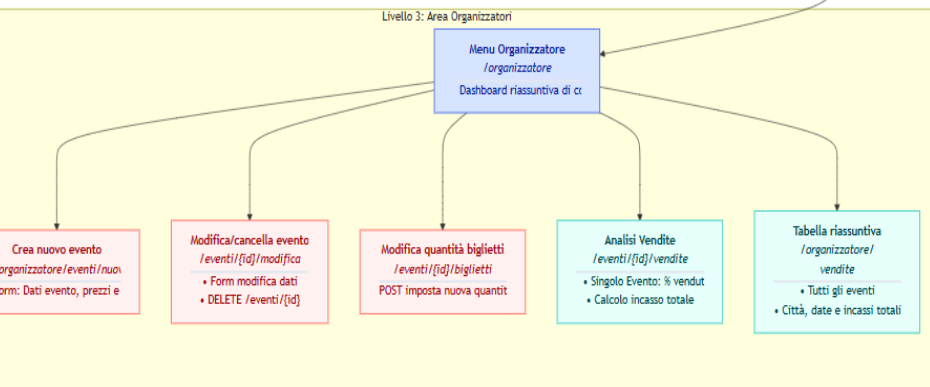
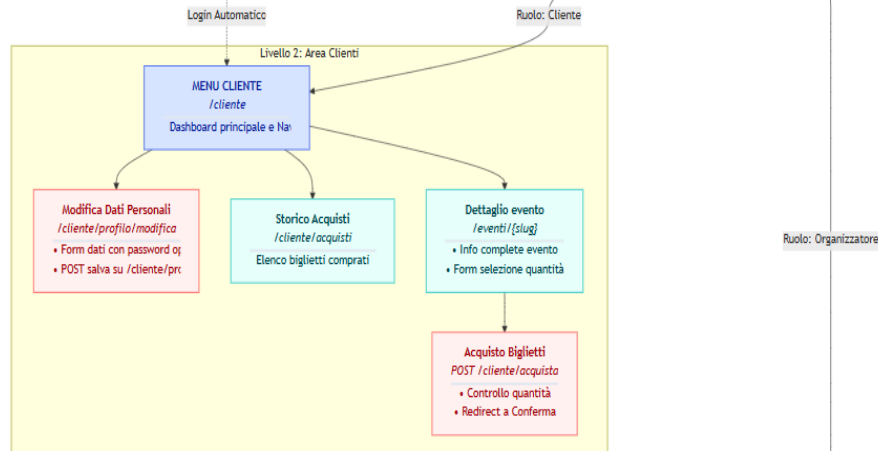
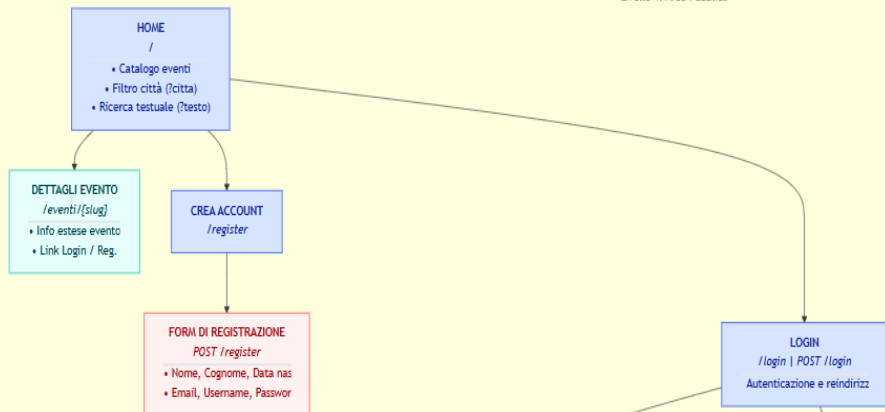
La struttura del database è definita tramite migration nella cartella [database/migrations](#). Le tabelle principali riguardano utenti, eventi, acquisti, partecipazioni e metodi di pagamento. Il popolamento iniziale è gestito da [database/seeder/DatabaseSeeder.php](#). Il seeder crea gli organizzatori, i clienti, i metodi di pagamento comuni e gli eventi. L'uso del seeder permette di ottenere un database pronto dopo l'esecuzione di `php artisan migrate:fresh --seed`. La connessione al database è definita in [config/database.php](#). Il progetto usa le variabili `$HOST`, `$DB`, `$USER` e `$PASSWORD`. La configurazione non dipende dal file `.env`. L'accesso al database avviene tramite Eloquent, quindi tramite il livello PDO di Laravel.

4.10 View Blade, CSS e JavaScript

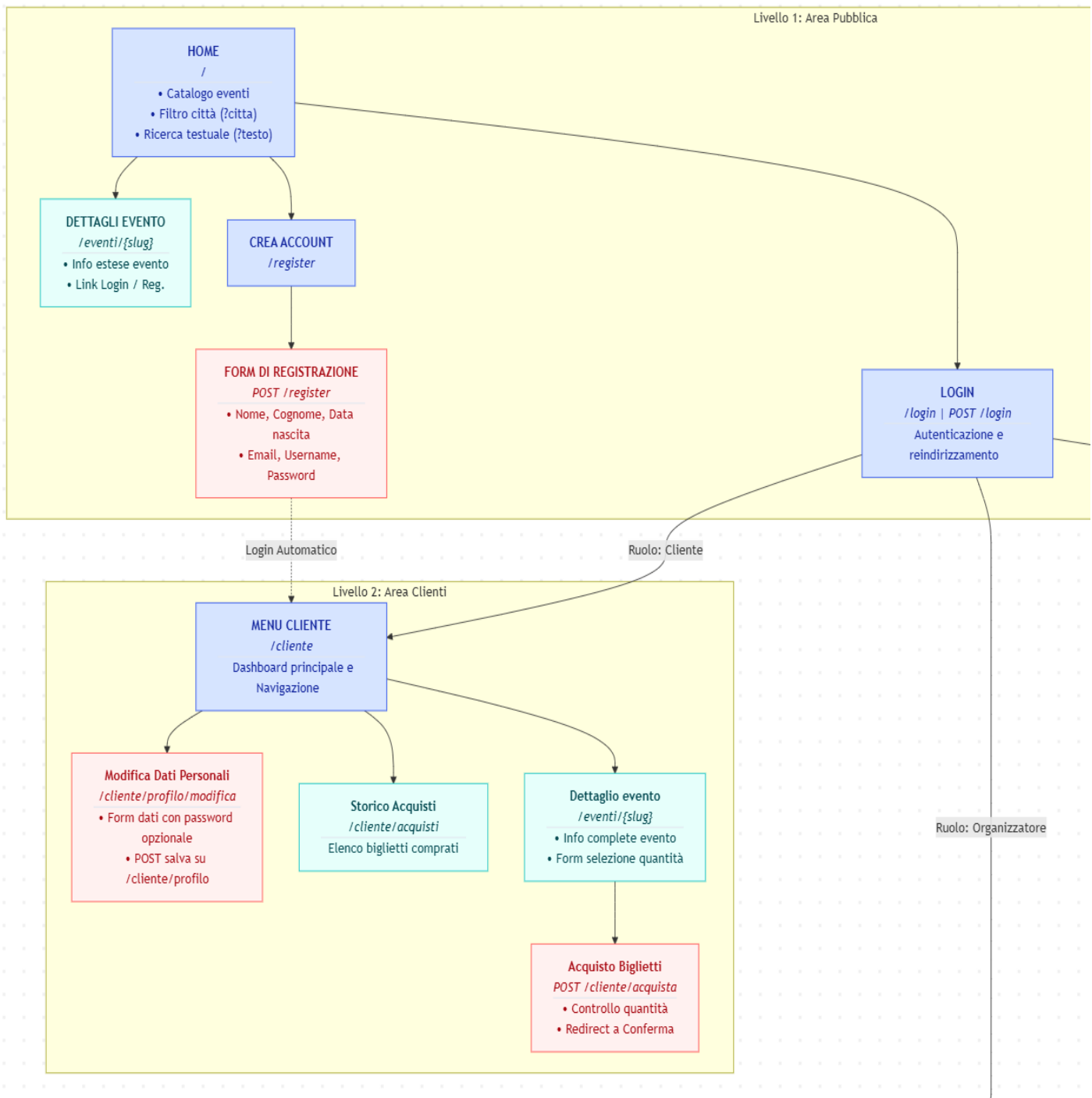
Le view sono organizzate per area funzionale: pubblico, cliente, organizzatore, amministratore, auth, layouts e pagination. I layout principali si trovano in [resources/views/layouts](#) e permettono di riutilizzare la struttura comune delle pagine. Il CSS principale si trova in [public/css/style.css](#). Le card vengono utilizzate solo per rappresentare gli eventi, mentre le altre sezioni del sito mantengono una struttura più semplice. Il codice JavaScript è centralizzato in [public/js/functions.js](#). In questo file sono presenti le funzioni jQuery per la gestione AJAX del "Parteciperò" e per il controllo lato client dello sconto last-minute. Il codice JavaScript non viene scritto inline nelle view, così da mantenere separati struttura HTML e comportamento dinamico.

5 Organization chart

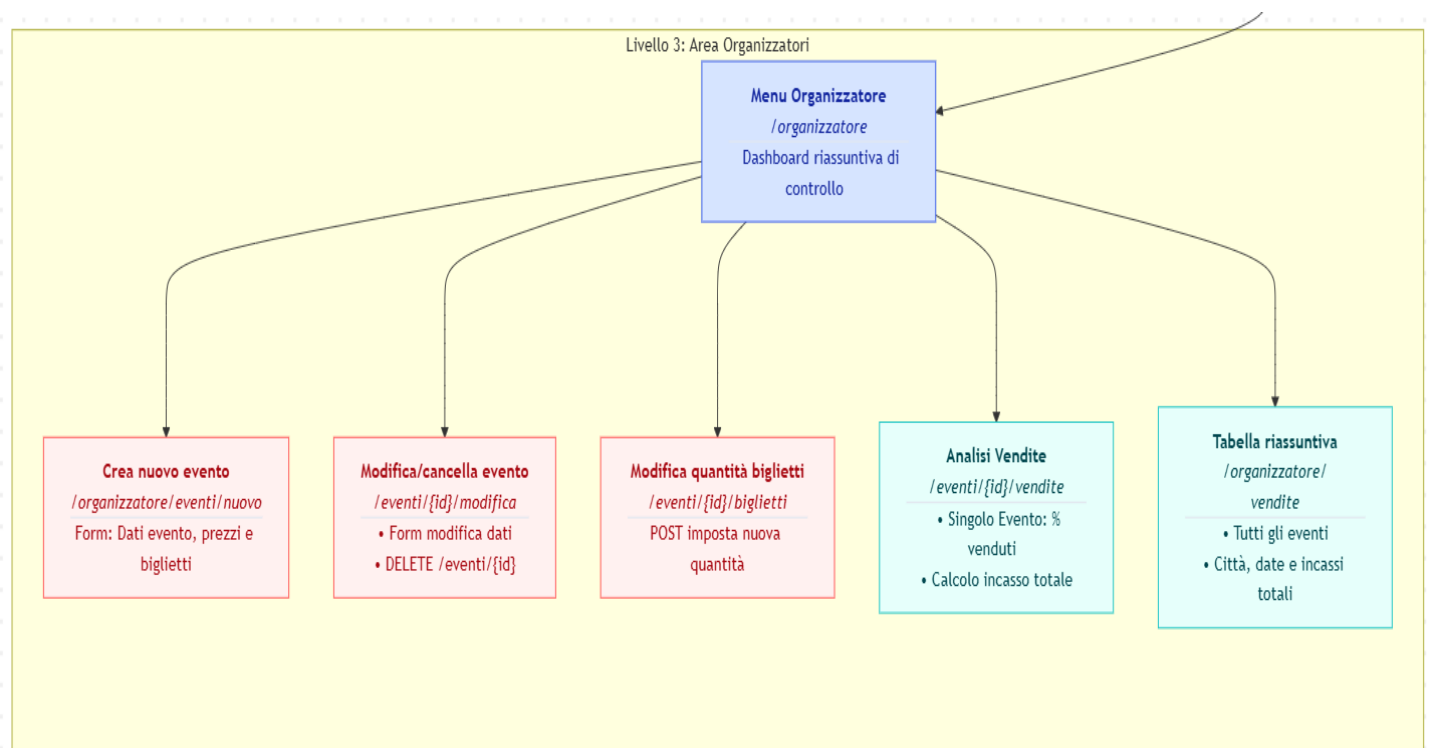
In questo capitolo viene presentato l'Organization Chart della piattaforma **MOSH PIT**, il quale illustra schematicamente l'alberatura delle pagine, la mappa del sito e l'architettura dei collegamenti (link) tra le diverse aree funzionali del sistema (pubblica, cliente, organizzatore e amministratore).



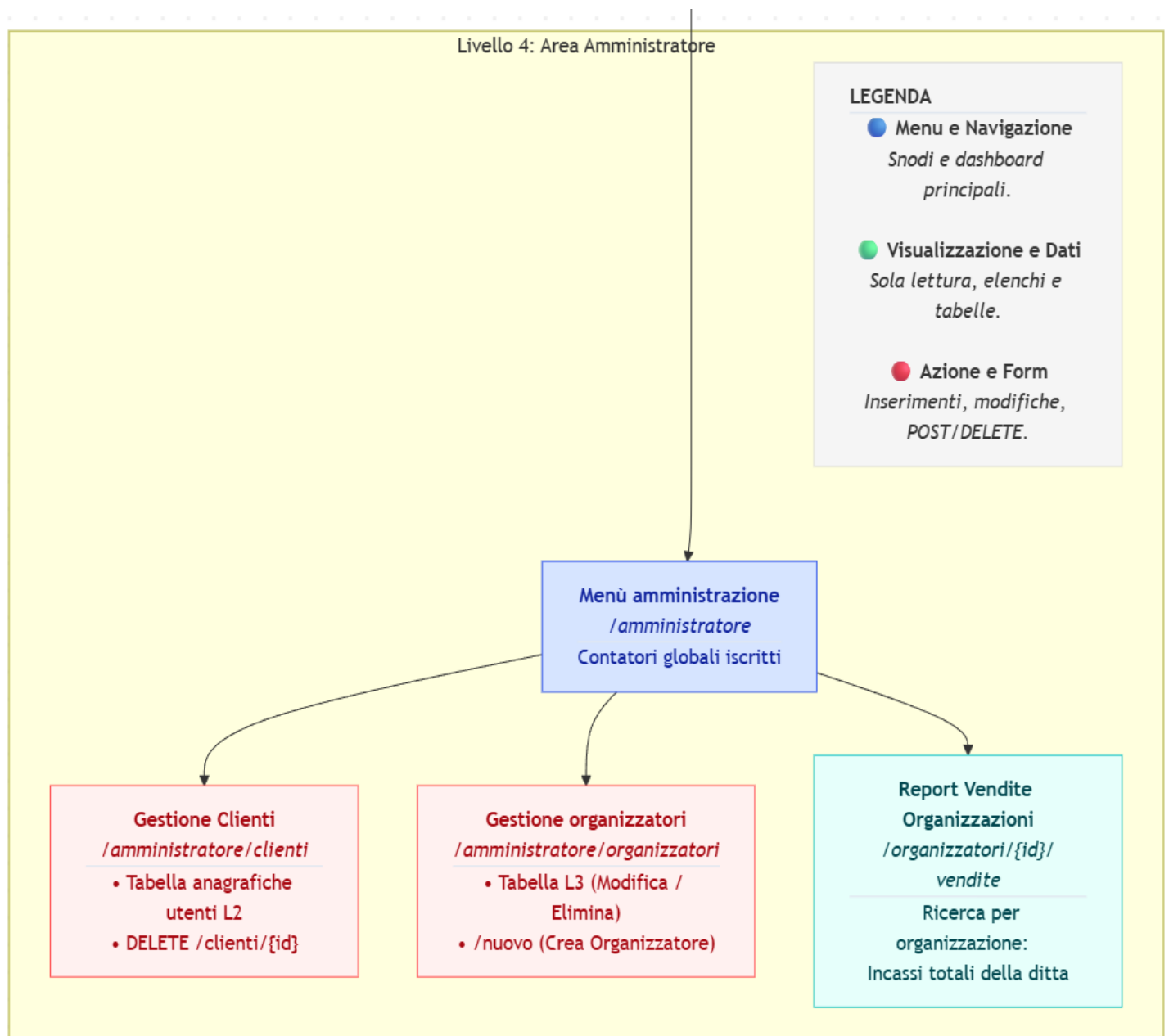
Schema dei link della piattaforma Moshpit



Ingrandimento Livello 1 e 2



Ingrandimento Livello 3



Ingrandimento Livello 4

6 Soluzioni adottate

In questa sezione vengono illustrate le soluzioni tecniche più significative sviluppate nel progetto Moshpit, con particolare attenzione alle parti implementate rispetto al progetto di riferimento laraProj6. Il progetto forniva una base Laravel con autenticazione, layout Blade, gestione del catalogo e un esempio di interazione AJAX tramite jQuery. Nel nostro progetto queste tecniche sono state riadattate e ampliate per gestire un dominio applicativo diverso: eventi, clienti, organizzatori, amministratori, acquisti, partecipazioni e sconti last-minute.

6.1 Gestione degli eventi futuri e mantenimento dello storico

Un evento passato viene considerato “eliminato”: non compare più nel catalogo pubblico, ma resta visibile a organizzatori e amministratori per lo storico e per gli incassi.

Nel PubblicoController, la query del catalogo pubblico filtra quindi gli eventi in base alla data:

```
1 $query = Evento::whereDate('data_evento', '>=', now()->toDateString()  
    )  
2     ->withCount('partecipazioni');
```

Questa istruzione significa che nella parte pubblica vengono mostrati solo gli eventi con data uguale o successiva alla data corrente. Anche nel dettaglio evento viene applicato lo stesso criterio:

```
1 $evento = Evento::where('slug', $slug)  
2     ->whereDate('data_evento', '>=', now()->toDateString())  
3     ->withCount('partecipazioni')  
4     ->firstOrFail();
```

In questo modo un utente non registrato o un cliente non può aprire manualmente l'URL di un evento passato. Al contrario, nell'area organizzatore non viene applicato questo filtro:

```
1 $eventi = Evento::where('organizzatore_id', auth()->id())  
2     ->withCount('partecipazioni')  
3     ->orderBy('data_evento', 'asc')  
4     ->paginate(8);
```

6.2 Ricerca testuale con wildcard

Il controllo sull'uso del carattere * è il seguente:

```
1 if (substr_count($testo, '*') > 1 ||
2     (strpos($testo, '*') !== false && substr($testo, -1) !== '*')) {
3     return redirect()
4         ->route('eventi.index')
5         ->withInput()
6         ->withErrors([
7             'testo' => 'Il carattere * è ammesso solo come ultimo
carattere della ricerca.',
8         ]);
9 }
```

Questo codice impedisce ricerche non valide come:

- *car
- ca*r
- c*a*

e ammette invece ricerche come: car*, concerto, Roma. Se il termine termina con *, viene rimossa la wildcard e viene costruita una query che cerca parole che iniziano con quel prefisso:

```
1 if (substr($testo, -1) === '*') {
2     $termine = substr($testo, 0, -1);
3     if ($termine === '') {
4         return redirect()
5             ->route('eventi.index')
6             ->withInput()
7             ->withErrors(['testo' => 'Inserisci almeno un carattere
prima del simbolo *.'],
8         );
9     }
10
11     $query->where(function ($q) use ($termine) {
12         $q->where('titolo', 'REGEXP', '[[[:<:]]' . $termine)
13             ->orWhere('descrizione', 'REGEXP', '[[[:<:]]' . $termine)
14             ->orWhere('citta', 'REGEXP', '[[[:<:]]' . $termine);
15     });
16 }
```

Se invece non c'è wildcard, la ricerca viene eseguita sulla parola intera:

```
1 $query->where(function ($q) use ($testo) {
```

```

2     $q->where('titolo', 'REGEXP', '[[[:<:]]' . $testo . '[[[:>:]]')
3         ->orWhere('descrizione', 'REGEXP', '[[[:<:]]' . $testo . '
        [[[:>:]]')
4         ->orWhere('citta', 'REGEXP', '[[[:<:]]' . $testo . '[[[:>:]]');
5     });

```

6.3 Funzionalità “Parteciperò” con jQuery e AJAX

Nella view del catalogo e nel dettaglio evento è presente una form tradicional:

```

1 <form
2     method="POST"
3     action="{{ route('eventi.partecipo', $evento->id) }}"
4     class="inline-form ajax-partecipero-form"
5     data-ajax-url="{{ route('eventi.partecipo.ajax', $evento->id) }}"
6     data-event-id="{{ $evento->id }}"
7 >
8     @csrf
9
10    <button type="submit" class="action-link partecipero-button">
11        Parteciperò
12    </button>
13 </form>

```

La route classica è:

```

1 Route::post('/eventi/{eventoId}/partecipo', [ClienteController::class
    , 'togglePartecipazione'])
2     ->name('eventi.partecipo')
3     ->middleware('auth')
4     ->middleware('can:isCliente');

```

La route AJAX dedicata è:

```

1 Route::post('/eventi/{eventoId}/partecipo/ajax', [ClienteController::
    class, 'togglePartecipazioneAjax'])
2     ->name('eventi.partecipo.ajax')
3     ->middleware('auth')
4     ->middleware('can:isCliente');

```

Entrambe le route sono protette da auth e can:isCliente. Nel file public/js/functions.js, jQuery intercetta il submit della form:

```
1 $( '.ajax-partecipero-form' ).on( 'submit', function (event) {
2     event.preventDefault();
3
4     var form = $(this);
5     var url = form.data( 'ajax-url' );
6     var eventId = form.data( 'event-id' );
7     var button = form.find( '.partecipero-button' );
8
9     $.ajax({
10         url: url,
11         type: 'POST',
12         data: form.serialize(),
13         dataType: 'json',
14
15         success: function (response) {
16             if (response.success) {
17                 $( '.partecipazioni-count[data-event-id="' + eventId +
18                 '"]' ).text( response.count );
19                 button.text( response.label );
20
21                 $( '#ajax-message' )
22                     .text( response.message )
23                     .show();
24             }
25             },
26         error: function () {
27             form.off( 'submit' );
28             form.submit();
29         }
30     });
31 });
```

Questa funzione usa jQuery in vari punti: (*'ajax-partecipero-form'*), (*this*), *form.data(...)*, *form.find(...)*, *form.serialize()*, *\$.ajax(...)*, *.text(...)*, *.show()*, *.off(...)*. Il controller restituisce una risposta JSON:

```
1 return response() -> json([
2     'success' => true,
3     'participated' => $utentePartecipa,
4     'count' => $conteggio,
5     'label' => $testoBottone,
6     'message' => $messaggio,
7 ]);
```


Il JavaScript usa poi questi dati per aggiornare il DOM:

```
1 $( '.partecipazioni-count[data-event-id="' + eventId + '"' ).text(  
    response.count);  
2 button.text(response.label);
```

6.4 Relazione tra eventi e partecipazioni

Per supportare la funzione “Parteciperò”, nel model Evento è stata definita una relazione Eloquent:

```
1 public function partecipazioni()  
2 {  
3     return $this->hasMany(Partecipazione::class, 'evento_id');  
4 }
```

Questa relazione permette di usare withCount('partecipazioni') nei controller:

```
1 $query = Evento::whereDate('data_evento', '>=', now()->toDateString()  
    )  
2     ->withCount('partecipazioni');
```

In questo modo Laravel aggiunge automaticamente il campo: \$evento->partecipazioni_count, che viene mostrato nella view:

```
1 <span class="partecipazioni-count" data-event-id="{{ $evento->id }}">  
2     {{ $evento->partecipazioni_count }}  
3 </span>
```

La scelta di usare withCount() è più pulita rispetto a calcolare manualmente il numero di partecipazioni dentro la view, che riceve già il dato pronto e si limita a visualizzarlo. Inoltre, lo stesso valore viene aggiornato tramite AJAX quando il cliente clicca su “Parteciperò”.

6.5 Sconto last-minute centralizzato nel model Evento

Lo sconto last-minute è stato implementato centralizzando la logica nel model Evento. Il prezzo scontato serve in più punti: nella card evento, nel dettaglio evento e durante l'acquisto. Nel model è presente un metodo che verifica se lo sconto è attivo:

```
1 public function scontoLastMinuteAttivo()  
2 {  
3     if ($this->percentuale_sconto_last_minute <= 0) {  
4         return false;  
5     }  
6  
7     $dataEvento = Carbon::parse($this->data_evento)->startOfDay();  
8     $dataInizioSconto = $dataEvento->copy()->subDays($this->  
9         giorni_attivazione_sconto);  
10  
11     return now()->startOfDay()->greaterThanOrEqualTo(  
12         $dataInizioSconto);  
13 }
```

Il prezzo effettivo viene calcolato in un secondo metodo:

```
1 public function prezzoEffettivo()  
2 {  
3     if (! $this->scontoLastMinuteAttivo()) {  
4         return $this->prezzo_biglietto;  
5     }  
6  
7     $sconto = ($this->prezzo_biglietto * $this->  
8         percentuale_sconto_last_minute) / 100;  
9  
10    return round($this->prezzo_biglietto - $sconto, 2);  
11 }
```

Questa soluzione evita di ripetere il calcolo in più file. Per esempio, nella view si può scrivere:

```
1 @if($evento->scontoLastMinuteAttivo())  
2     <p>  
3         <strong>Prezzo:</strong>  
4         <span class="old-price">€ {{ number_format($evento->  
5             prezzo_biglietto, 2, ',', '.') }}</span>  
6         <span class="discount-price">€ {{ number_format($evento->  
7             prezzoEffettivo(), 2, ',', '.') }}</span>  
8         <span class="discount-percent">(-{{ $evento->  
9             percentuale_sconto_last_minute }}%)</span>  
10    </p>  
11 @else
```

```

9      <p>
10         <strong>Prezzo:</strong>
11         € {{ number_format($evento->prezzo_biglietto, 2, ',', '.') }}
12     </p>
13 @endif

```

Nel ClienteController, invece, lo stesso metodo viene usato per calcolare il totale dell'acquisto:

```

1 $prezzoEffettivo = $evento->prezzoEffettivo();
2
3 $acquisto = Acquisto::create([
4     'utente_id' => auth()->id(),
5     'evento_id' => $evento->id,
6     'metodo_pagamento_id' => $metodoPagamento->id,
7     'quantita' => $quantita,
8     'prezzo_totale' => $prezzoEffettivo * $quantita,
9     'data_acquisto' => now(),
10 ]);

```

6.6 Controllo JavaScript dello sconto last-minute

Oltre alla validazione lato server, è stato aggiunto un controllo lato client nel file public/js/functions.js. La funzione usa jQuery per recuperare i campi della form:

```

1 var form = $(' [data-event-form="true"] ');
2
3 var discountInput = $('#percentuale_sconto_last_minute');
4 var daysInput = $('#giorni_attivazione_sconto');
5 var dateInput = $('#data_evento');

```

Il codice è collegato alla view tramite un attributo HTML:

```

1 <form method="POST" action="{{ route('organizzatore.eventi.salva') }}"
  " data-event-form="true">

```

Questa soluzione evita di scrivere JavaScript inline nella view. La view espone solo un attributo, mentre il comportamento viene definito in functions.js. La funzione calcola i giorni mancanti all'evento:

```

1 function calculateDaysUntilEvent() {
2     if (!dateInput.val()) {
3         return null;
4     }
5
6     var today = new Date();
7     today.setHours(0, 0, 0, 0);
8
9     var eventDate = new Date(dateInput.val() + 'T00:00:00');
10    if (Number.isNaN(eventDate.getTime())) {
11        return null;
12    }
13
14    return Math.round((eventDate.getTime() - today.getTime()) /
15                      86400000);
16 }

```

Poi confronta il valore inserito dall'organizzatore:

```

1 if (daysUntilEvent !== null && days > daysUntilEvent) {
2     showError('I giorni non possono essere maggiori dei giorni che
3     mancano all evento. Massimo: ' + daysUntilEvent + '.');
4     return;
5 }

```

6.7 Gestione dello sconto in modifica evento

Un caso particolare riguarda la modifica di un evento con sconto già impostato. Supponiamo che l'organizzatore imposti uno sconto 10 giorni prima dell'evento. Il giorno dopo mancheranno 9 giorni, quindi il valore salvato non sarebbe più valido se venisse ricontrollato come nuovo input. Per evitare questo problema, nel `OrganizzatoreController` è stato aggiunto un controllo che verifica se i campi dello sconto sono stati davvero modificati:

```

1 $scontoModificato =
2     $request->input('data_evento') != $evento->data_evento ||
3     (int) $request->input('percentuale_sconto_last_minute') != (int)
4     $evento->percentuale_sconto_last_minute ||
5     (int) $request->input('giorni_attivazione_sconto') != (int)
6     $evento->giorni_attivazione_sconto;

```

La validazione dello sconto viene eseguita solo se uno di questi campi è cambiato:

```
1 if ($scontoModificato) {
2     $erroreSconto = $this->controllaScontoLastMinute(
3         $request->input('data_evento'),
4         (int) $request->input('percentuale_sconto_last_minute'),
5         (int) $request->input('giorni_attivazione_sconto')
6     );
7     if ($erroreSconto !== null) {
8         return redirect()
9             ->back()
10            ->withInput()
11            ->withErrors([
12                'giorni_attivazione_sconto' => $erroreSconto,
13            ]);
14     }
15 }
```

Questa è una soluzione specifica del progetto, nata da un caso reale di utilizzo. Evita che l'organizzatore venga bloccato quando modifica campi non collegati allo sconto, come titolo, descrizione o luogo.

6.8 Gestione immagini eventi

In Moshpit ogni evento ha un'immagine di riferimento. Abbiamo scelto di salvare nel campo immagine solo il nome del file:

```
1 'immagine' => 'placeholder.png',
```

Le immagini vengono collocate in: public/images/eventi. Nelle view vengono caricate così:

```
1 @if($evento->immagine)
2     titolo }}"
5         class="event-card-image"
6     >
7 @else
8     
13 @endif
```

Inoltre, l'uso di placeholder.png nel seeder garantisce che ogni evento abbia sempre un'immagine visualizzabile, evitando card vuote o immagini mancanti. Nel CSS sono state create classi dedicate:

```
1 .event-image-box {
2     width: 100%;
3     height: 170px;
4     margin-bottom: 12px;
5     overflow: hidden;
6     background: #e9e9e9;
7     border-radius: 6px;
8 }
9
10 .event-card-image {
11     display: block;
12     width: 100%;
13     height: 100%;
14     object-fit: cover;
15 }
```

La proprietà object-fit: cover permette di mantenere proporzioni corrette e riempire lo spazio della card senza deformare l'immagine.

6.9 Conferma acquisto

Nel controller, dopo l'acquisto, l'utente viene mandato alla route di conferma:

```
1 return redirect()->route('cliente.acquisto.conferma', $acquisto->id);
```

La view conferma-acquisto.blade.php mostra un riepilogo leggendo i dati dell'acquisto e delle relazioni collegate:

```
1 <h3>Grazie!</h3>
2
3 <p>Il tuo acquisto è stato registrato correttamente.</p>
```

E successivamente la tabella con il riepilogo dell'acquisto. Nel controller la conferma viene recuperata filtrando anche sull'utente autenticato:

```
1 $acquisto = Acquisto::with(['evento', 'metodoPagamento'])
2     ->where('utente_id', auth()->id())
3     ->where('id', $acquistoId)
4     ->firstOrFail();
```

Questo impedisce a un cliente di visualizzare la conferma di un acquisto appartenente a un altro utente semplicemente modificando l'id nell'URL.

6.10 Metodi di pagamento comuni

La tabella metodi_pagamento è stata semplificata. La migration contiene solo:

```
1 Schema::create('metodi_pagamento', function (Blueprint $table) {
2     $table->bigIncrements('id');
3     $table->string('nome', 80);
4     $table->timestamps();
5 });
```

Il model è altrettanto semplice:

```
1 class MetodoPagamento extends Model
2 {
3     protected $table = 'metodi_pagamento';
4     protected $fillable = [
5         'nome',
6     ];
7 }
```

Nel seeder vengono creati una sola volta i metodi:

```
1 $metodoCarta = MetodoPagamento::create([
2     'nome' => 'Carta',
3 ]);
4 $metodoPayPal = MetodoPagamento::create([
5     'nome' => 'PayPal',
6 ]);
7
8 $metodoBonifico = MetodoPagamento::create([
```

```
9     'nome' => 'Bonifico',  
10 ];
```

Nel controller viene verificato solo che il metodo scelto esista:

```
1 $metodoPagamento = MetodoPagamento::findOrFail($request->input('metodo_pagamento_id'));
```

6.11 Protezione CSRF e metodo DELETE

Tutte le form che modificano dati usano il token CSRF:

```
1 @csrf
```

Per esempio, la form “Parteciperò” contiene:

```
1 <form method="POST" action="{ route('eventi.partecipo', $evento->id) } }">  
2     @csrf  
3     <button type="submit">Parteciperò</button>  
4 </form>
```

Questo è importante anche per AJAX, perché `form.serialize()` invia anche il token CSRF al server. Per le eliminazioni viene usato:

```
1 @method('DELETE')
```

per esempio:

```
1 <form method="POST" action="{ route('organizzatore.eventi.elimina', $evento->id) } }" class="inline-form">  
2     @csrf  
3     @method('DELETE')  
4  
5     <button type="submit" class="action-link danger-link">
```



```
6         Elimina
7     </button>
8 </form>
```

Il browser invia una POST, ma Laravel interpreta la richiesta come DELETE.

6.12 Configurazione database

In config/database.php non viene usato .env per la connessione, ma vengono caricate le variabili del file connect.php:

```
1 require(__DIR__ . '/../../include/connect.php');
```

La connessione di default è impostata direttamente:

```
1 'default' => 'mariadb',
```

e usa le variabili:

```
1 'host' => $HOST,
2 'database' => $DB,
3 'username' => $USER,
4 'password' => $PASSWORD,
```

6.13 Organizzazione CSS senza framework esterni

Il CSS è scritto nel file: public/css/style.css. Sono state definite classi specifiche per il dominio del progetto, ad esempio:

```
1 .event-grid {
2     display: grid;
3     grid-template-columns: repeat(auto-fill, minmax(285px, 1fr));
4     gap: 18px;
5     width: 100%;
```

```
6     margin: 18px 0;
7 }
```

Questa regola crea automaticamente una griglia responsive per le card evento. Le card sono state limitate agli eventi:

```
1 .event-card {
2     background: #ffffff;
3     border: 1px solid #d6d6d6;
4     border-radius: 8px;
5     padding: 18px;
6 }
```

Anche le azioni sono state rese testuali e uniformi:

```
1 .action-link,
2 button.action-link {
3     display: inline;
4     background: none;
5     border: none;
6     color: #5f2380;
7     text-decoration: underline;
8     padding: 0;
9     margin: 0 18px 0 0;
10    font: inherit;
11    cursor: pointer;
12    border-radius: 0;
13    box-shadow: none;
14 }
```

6.14 Home pubblica con iframe Google Maps

Nella home è stato inserito un iframe di Google Maps per indicare la localizzazione di riferimento del servizio:

```
1 <iframe
2     src="https://www.google.com/maps?q=Universit%C3%A0%20Politecnica
3     %20delle%20Marche%20Ingegneria%20Ancona&output=embed"
4     width="100%"
5     height="360"
```

```
5     style="border:0;"
6     allowfullscreen
7     loading="lazy"
8     referrerpolicy="no-referrer-when-downgrade">
9 </iframe>
```

È stato usato `loading="lazy"` per evitare il caricamento immediato della mappa se non necessario.

7 Conclusione

La progettazione e l'implementazione della piattaforma **MOSHPIT** hanno permesso di mettere in pratica con successo le metodologie e le tecnologie analizzate durante il corso di *Tecnologie Web*. Il risultato ottenuto è un'applicazione solida, funzionale e rispondente a tutti i requisiti, sia standard sia opzionali, delineati nelle specifiche d'esame.

L'adozione del framework **Laravel 12** si è rivelata determinante: l'architettura Model-View-Controller (MVC), l'efficacia dell'ORM Eloquent nella gestione del database e i meccanismi nativi di sicurezza (come la protezione CSRF e l'autenticazione) hanno velocizzato il ciclo di sviluppo, garantendo al contempo un codice pulito, manutenibile e scalabile. Parallelamente, l'integrazione di tecnologie front-end dinamiche tramite JavaScript e jQuery ha consentito di offrire un'experience utente fluida, reattiva e coerente.

Al di là degli aspetti prettamente tecnici, l'esperienza si è dimostrata altamente formativa dal punto di vista metodologico e professionale. Il costante confronto all'interno del team e la scelta di mantenere un carico di lavoro perfettamente bilanciato hanno evidenziato l'importanza delle dinamiche di *cooperative working*. La necessità di coordinarsi passo dopo passo ha stimolato la capacità di *problem solving* collettivo, riflettendo fedelmente i flussi di lavoro tipici dei moderni contesti aziendali di sviluppo software.